5.3 Self-healing Infrastructure: Budowa agentów monitorujących logi i automatycznie sugerujących poprawki w infrastrukturze AWS

Moduł 5: Agenty AI w procesach CI/CD i DevOps

Krzysztof Rytlewski · 2026

Czym jest self-healing infrastructure?

Self-healing infrastructure (samonaprawiająca się infrastruktura) to podejście, w którym system automatycznie wykrywa anomalie lub awarie, a następnie podejmuje działania naprawcze — bez interwencji człowieka lub informując go o gotowej sugestii poprawki.

Tradycyjny model reagowania na incydenty:

1. System generuje alert.
2. Inżynier odczytuje alert (być może w środku nocy).
3. Inżynier analizuje logi i wdraża poprawkę.

W modelu self-healing agent AI przejmuje kroki 2 i 3 — analizuje logi, identyfikuje przyczynę i albo **automatycznie stosuje poprawkę** (dla bezpiecznych, przewidywalnych scenariuszy), albo **generuje gotową do zatwierdzenia sugestię** (dla zmian wymagających nadzoru).

Usługi AWS biorące udział w self-healing

Usługa AWS	Rola w self-healing
CloudWatch Logs	Źródło danych — logi aplikacji i infrastruktury
CloudWatch Alarms	Wyzwalacz — alert przy przekroczeniu progu metryki
SNS	Kanał — dostarcza powiadomienie do Lambda lub email
Lambda	Wykonawca — uruchamia agenta w odpowiedzi na alarm
AWS SDK	Narzędzie — agent wykonuje akcje naprawcze (restart, skalowanie)
GitHub API	Kanał sugestii — agent otwiera PR z poprawką konfiguracji

Architektura agenta self-healing — przepływ zdarzeń

```
[AWS CloudWatch Logs]
        │  logi Lambda (błędy, timeouty, cold starty)
        ▼
[CloudWatch Metric Filter]
        │  filtruje zdarzenia: "Task timed out", "Error:"
        ▼
[CloudWatch Alarm]
        │  próg: >5 błędów w ciągu 5 minut
        ▼
[SNS Topic]
        │  wiadomość JSON z informacją o alarmie
        ▼
[Lambda — agent self-healing]
        │
        ├─ 1. Pobierz ostatnie logi (CloudWatch Logs API)
        ├─ 2. Wyślij logi do LLM (OpenAI) z prośbą o diagnozę
```

Podział na **działania automatyczne** (3a) i **sugestie** (3b) jest kluczowy dla bezpieczeństwa.

Pobieranie i analiza logów CloudWatch przez Node.js

```
import { CloudWatchLogsClient, FilterLogEventsCommand }
  from '@aws-sdk/client-cloudwatch-logs';

const logsClient = new CloudWatchLogsClient({ region: 'eu-west-1' });

export async function getRecentErrors(logGroupName, minutesBack = 30) {
  const startTime = Date.now() - minutesBack * 60 * 1000;
  const response = await logsClient.send(new FilterLogEventsCommand({
    logGroupName, startTime,
    filterPattern: '?ERROR ?Error ?error ?"Task timed out"',
    limit: 50,
  }));
  return response.events?.map(e => ({
    timestamp: new Date(e.timestamp).toISOString(),
    message: e.message.trim(),
```

Filtr `?ERROR ?Error ?error` to składnia CloudWatch Logs — `?` oznacza „zawiera to słowo". Limit 50 chroni przed nadmiernym zużyciem tokenów LLM.

Diagnoza i sugestia poprawki przez LLM

Rdzeń agenta to wysłanie zebranych logów do LLM z prompem zawierającym kontekst infrastruktury. Prompt definiuje cztery pola odpowiedzi w JSON:

- `summary` — krótkie podsumowanie problemu,
- `rootCause` — opis przyczyny,
- `category` — `"config"` (zmiana konfiguracji Lambda), `"code"` (błąd w kodzie) lub `"external"` (MongoDB, OpenAI),
- `autoFix` — obiekt `{ type: "timeout|memory", value: "..." }` lub `null`,
- `codeSuggestion` — opis poprawki kodu lub `null`.

Parametr `response_format: { type: 'json_object' }` gwarantuje parsowalne JSON w odpowiedzi OpenAI. Konfiguracja monitorowanej funkcji Lambda (timeout, pamięć, runtime) trafia do promptu jako kontekst.

Automatyczne stosowanie bezpiecznych poprawek konfiguracyjnych

Gdy LLM wskaże kategorię `config`, agent może automatycznie zaktualizować konfigurację Lambda przez AWS SDK:

```
import { LambdaClient, UpdateFunctionConfigurationCommand }
  from '@aws-sdk/client-lambda';

const lambdaClient = new LambdaClient({ region: 'eu-west-1' });

export async function applyConfigFix(functionName, autoFix) {
  const update = {};
  if (autoFix.type === 'timeout') {
    update.Timeout = Math.min(parseInt(autoFix.value), 900); // max 900s
  } else if (autoFix.type === 'memory') {
    update.MemorySize = Math.min(parseInt(autoFix.value), 10240); // max 10 GB
  }
  if (!Object.keys(update).length) return null;
  await lambdaClient.send(
    new UpdateFunctionConfigurationCommand({ FunctionName: functionName, ...update })
```

Zasada minimalnych uprawnień dla agenta IAM

Rola IAM agenta musi mieć dostęp wyłącznie do niezbędnych akcji na konkretnych zasobach:

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": [
      "lambda:UpdateFunctionConfiguration",
      "lambda:GetFunctionConfiguration",
      "logs:FilterLogEvents",
      "logs:DescribeLogGroups"
    ],
    "Resource":
      "arn:aws:lambda:eu-west-1:123456789:function:ai-app-backend"
  }]
```

Nigdy nie nadawaj agentowi uprawnień `*` ani `lambda:*` — ograniczaj do konkretnych akcji i konkretnych zasobów ARN. Naruszenie tej zasady może prowadzić do niekontrolowanych zmian w infrastrukturze produkcyjnej.

Sugestie poprawek kodu — automatyczny Pull Request na GitHubie

Gdy LLM zidentyfikuje błąd w kodzie (kategoria `code`), agent otwiera Pull Request z opisem diagnozy i sugestią poprawki zamiast samodzielnie modyfikować kod produkcyjny.

Agent tworzy nową gałąź `fix/self-healing-<timestamp>`, a następnie przez `octokit.pulls.create()` otwiera PR z tytułem zawierającym podsumowanie problemu i treścią opisującą przyczynę oraz sugestię poprawki wygenerowaną przez LLM.

Taki PR trafia do kolejki przeglądu — deweloper widzi diagnozę i sugestię, może ją odrzucić lub zaimplementować. Bot code review z sekcji 5.2 może automatycznie przejrzeć proponowane zmiany.

Wdrożenie agenta jako Lambda wyzwalanej przez SNS

Cały agent działa jako osobna funkcja Lambda subskrybująca temat SNS powiązany z alarmem CloudWatch.

Funkcja `handler` wykonuje cztery kroki: pobiera logi z CloudWatch przez `getRecentErrors()`, pobiera konfigurację monitorowanej funkcji Lambda przez AWS SDK, wysyła logi do LLM przez `diagnoseAndSuggest()`, a następnie — zależnie od kategorii diagnozy — albo automatycznie stosuje poprawkę konfiguracyjną przez `applyConfigFix()`, albo otwiera PR przez `createSuggestionPR()`, albo rejestruje problem zewnętrzny.

Wynik każdego uruchomienia agenta jest logowany strukturalnie do CloudWatch dla późniejszej analizy skuteczności.

Podsumowanie — wartość architektury self-healing

Self-healing infrastructure przynosi trzy kluczowe korzyści operacyjne:

1. **Szybszy czas reakcji** — agent analizuje logi i stosuje bezpieczne poprawki w ciągu sekund, bez czekania na inżyniera.

2. **Ograniczenie alert fatigue** — rutynowe problemy (np. timeout Lambda) są naprawiane automatycznie, a inżynierowie dostają powiadomienia tylko o złożonych awariach.

3. **Dokumentowana historia napraw** — każde działanie agenta jest logowane, PR zawiera opis przyczyny i zastosowanej poprawki, co ułatwia analizę post-mortem.

Kluczowe ograniczenie: agent musi mieć minimalny zakres uprawnień — nigdy nie nadajemy mu pełnego dostępu do infrastruktury.